10. Работа с реляционными базами данных на примере PostgreSQL

Введение в реляционные базы данных

Реляционная база данных — это способ организации данных в виде таблиц с установленными связями между ними.

Основные понятия:

- **Таблица** — структура для хранения данных
- **Строка (запись)** — отдельный элемент данных
- **Столбец (поле)** — атрибут записи
- **Первичный ключ** — уникальный идентификатор записи
- **Внешний ключ** — ссылка на запись в другой таблице

PostgreSQL

PostgreSQL — объектно-реляционная система управления базами данных с открытым исходным кодом.

Преимущества PostgreSQL:

- Высокая производительность
- ACID-совместимость
- Расширяемость
- Поддержка JSON
- Мощные индексы
- Репликация

Установка PostgreSQL:

```
# macOS
brew install postgresql
brew services start postgresql

# Ubuntu/Debian sudo
apt update
sudo apt install postgresql postgresql-contrib


# Docker
docker run --name postgres -e POSTGRES_PASSWORD=password -p 5432:5432 -d
postgres
```

Подключение к PostgreSQL из Go

Основные драйверы для PostgreSQL:

- `pq` — чистый Go драйвер
- `pgx` — современный высокопроизводительный драйвер

Установка драйвера:

```
go mod init
database-example
go get
github.com/lib/pq #
```

или

```
go get
github.com/jackc/pgx/v5
```

Базовое подключение:

```
package main

import (
   "database/sql" "fmt"
   "log"
   _ "github.com/lib/pq"
)

func main() {
   // Строка подключения
   connStr := "user=username dbname=mydb sslmode=disable password=mypassword
   host=localhost port=5432"
   // Подключение к базе данных
   db, err := sql.Open("postgres", connStr)
   if err != nil {
      log.Fatal(err)
   }
   defer db.Close()
   // Проверка соединения
   err = db.Ping()
   if err != nil {
      log.Fatal(err)
   }
}
```

Создание таблиц и основные операции

Создание таблицы:

```
func createTable(db *sql.DB)
   error { query := `
   CREATE TABLE IF NOT EXISTS users (
      id SERIAL PRIMARY KEY,
      name VARCHAR(100) NOT NULL,
      email VARCHAR(100) UNIQUE NOT NULL,
      age INTEGER,
      created_at TIMESTAMP DEFAULT
      CURRENT_TIMESTAMP
   )`

   _, err :=
   db.Exec(query)
   return err
}
```

Структура для работы с данными:

```
type User
struct {
ID    int
Name  `db:"id"`  string
Emai  `db:"name"`
l     string
Age   `db:"email"`  int
      `db:"age"`
CreatedAt time.Time
`db:"created_at"`
}
```


MERION

CRUD операции

```
type UserRepository
   struct { db
   *sql.DB
}
```

CREATE — Создание записей

```
func (r UserRepository) Create(ctx context.Context, name, email string,
   age int) (int, error) { var id int
   query := `
      INSERT INTO users (name,
      email, age) VALUES ($1, $2,
      $3)
      RETURNING id`

   err := db.QueryRow(query, name, email,
   age).Scan(&id) if err != nil {
      return 0, err
   }

   return id, nil
}
```

READ — Чтение записей

Получение одной записи:

```
func (r UserRepository) GetByID(ctx context.Context, id int) (*User, error) {
   user := &User{}
   query := `
      SELECT id, name, email, age, created_at
      FROM users WHERE id = $1`

   err := db.QueryRow(query, id).Scan(
     &user.ID,
      &user.Name,
      &user.Email, &user.Age,
      &user.CreatedAt,
   )

   if err != nil {
    return nil, err
   }

    return user, nil
}
```

READ ALL— Чтение всех записей

```
func (r UserRepository) GetAll(ctx context.Context) ([]User, error) { var
   users []User
   query := `
      SELECT id, name, email, age, created_at FROM users
      ORDER BY created_at DESC`

   rows, err := db.Query(query) if err != nil {
      return nil, err
   }
   defer rows.Close()

   for rows.Next() {
   var user User
      err := rows.Scan(
       &user.ID, &user.Name, &user.Email,
         &user.Age, &user.CreatedAt,
      )
      if err != nil { return nil, err
      }
      users = append(users, user)
   }

   return users, nil
}
```

UPDATE — Обновление записей

```
func (r UserRepository) Update(
ctx context.Context, id int, name, email string, age int) error {
query := `
      UPDATE users
      SET name = $2, email = $3, age = $4
      WHERE id = $1`

   result, err := db.Exec(query, id, name, email, age)
    if err != nil {
      return err
   }

   rowsAffected, err := result.RowsAffected()
   if err != nil {
      return err
   }

   if rowsAffected == 0 {
      return fmt.Errorf("пользователь с ID %d не найден", id)
   }

   return nil
}
```

DELETE — Удаление записей

```
func (r UserRepository) Delete(ctx context.Context, id int) error
   {
query := `DELETE FROM users WHERE id = $1`

   result, err := db.Exec(query, id)
   if err != nil {
      return err
   }

   rowsAffected, err := result.RowsAffected()
   if err != nil {
      return err
   }

   if rowsAffected == 0 {
      return fmt.Errorf("пользователь с ID %d не найден", id)
   }

   return nil
}
```

Работа с транзакциями

Транзакция обеспечивает атомарность операций — либо все операции выполняются успешно, либо все отменяются.

```
func transferMoney(db *sql.DB, fromUserID, toUserID int, amount float64) error { tx,
    err := db.Begin()  // Начинаем транзакцию
    if err != nil {
        return err
    }
    defer func() {
        if err != nil {
            tx.Rollback() // Откатываем транзакцию в случае ошибки
        }
    }()

    // Списываем деньги с первого аккаунта
    _, err = tx.Exec(
        "UPDATE accounts SET balance = balance - $1 WHERE user_id = $2",
        amount, fromUserID,
    )
    if err != nil {
        return err
    }
    // Зачисляем деньги на второй аккаунт
    _, err = tx.Exec(
        "UPDATE accounts SET balance = balance + $1 WHERE user_id = $2",
        amount, toUserID,
    )
    if err != nil {
        return err
    }
    return tx.Commit() // Подтверждаем транзакцию
}
```

Подготовленные запросы

Подготовленный запрос разделяет **код** (структуру SQL-запроса) и **данные** (пользовательский ввод).

Как это работает? Процесс выполнения запроса делится на два этапа:

1. **Подготовка (Preparation):** БД получает шаблон запроса, проводит анализ синтакса, проверяет права доступа, оптимизирует и составляет план выполнения запроса и кеширует его.
2. **Выполнение (Execution):** Серверу передаются данные, которые нужно подставить в готовый и оптимизированный план.

```
func getUsersByAge(db *sql.DB, minAge, maxAge int) ([]User, error) {
   // Подготавливаем запрос stmt, err :=
   db.Prepare(`
      SELECT id, name, email, age, created_at
      FROM users
      WHERE age BETWEEN $1 AND $2
      ORDER BY age`)
   if err != nil { return nil, err
   }
   defer stmt.Close()

   // Выполняем запрос
   rows, err := stmt.Query(minAge, maxAge)
   if err != nil {
      return nil, err
   }
   defer rows.Close()

   var users []User for rows.Next() {
      var user User
      err := rows.Scan(&user.ID, &user.Name, &user.Email, &user.Age, &user.CreatedAt)
      if err != nil {
      return nil, err
      }
      users = append(users, user)
   }

   return users, nil
}
```

Обработка ошибок

Типичные ошибки и их обработка:

```
func handleDatabaseErrors(err error) {
if err == sql.ErrNoRows {
      fmt.Println("Запись не найдена")
      return
   }

   if pqErr, ok := err.(*pq.Error); ok {
   switch pqErr.Code {
      case "23505": // unique_violation
         fmt.Println("Нарушение уникальности")
      case "23503": // foreign_key_violation
         fmt.Println("Нарушение внешнего ключа")
      case "23502": // not_null_violation
         fmt.Println("Нарушение NOT NULL")
      default:
         fmt.Printf("Ошибка базы данных: %s\n", pqErr.Code)
      }
      return
   }

   fmt.Printf("Неизвестная ошибка: %v\n", err)
}
```

Миграции базы данных

Миграции базы данных — это система контроля версий для вашей схемы БД, которая позволяет безопасно и предсказуемо вносить в нее изменения, а при необходимости — легко откатывать их, обеспечивая синхронизацию структуры данных между средой разработки, тестирования и продакшена.

Шаг 1: Установка Goose

Сначала вам нужно установить инструмент `goose` . Откройте терминал и выполните следующую команду:

```
go install github.com/pressly/goose/cmd/goose@latest
```

Эта команда скачает и установит последнюю версию `goose` в ваш каталог `GOPATH` (обычно `~/go/bin`).

Шаг 2: Инициализация миграций

Перейдите в директорию вашего Golang проекта и инициализируйте `goose`. Это создаст директорию для хранения ваших миграционных файлов (например, `database/migrations`):

```
goose -dir database/migrations init
```

- `-dir database/migrations` : Указывает путь к директории, где будут храниться миграции.

Шаг 3: Создание новой миграции

Чтобы создать новую миграцию, используйте команду `create` . Например, для создания таблицы пользователей:

```
goose -dir database/migrations create create_users_table sql
```

- `create create_users_table` : Создаст SQL-файл миграции с названием `XXXXXXX_create_users_table.sql` , где `XXXXXXX` - это временная метка.
- Файл будет содержать два пустых блока `up` (для применения изменений) и `down` (для отката изменений).

Шаг 4: Применение миграции

После того как вы добавили SQL-код в файлы миграции, вы можете применить их к

```
goose -dir database/migrations up
```

Эта команда применит все миграции, которые еще не были применены, к вашей базе данных.

Лучшие практики

1. **Всегда используйте параметризованные запросы** для защиты от SQL-инъекций
2. **Закрывайте соединения и результаты** с помощью `defer`
3. **Используйте транзакции** для связанных операций
4. **Настраивайте пул соединений** согласно нагрузке
5. **Обрабатывайте специфические ошибки** базы данных
6. **Используйте индексы** для ускорения запросов
7. **Проводите миграции** для изменения схемы
8. **Логируйте SQL-запросы** в режиме разработки

Альтернативы: ORM и Query Builders

GORM — популярная ORM:

```
// Установка
go get -u gorm.io/gorm
go get -u gorm.io/driver/postgres

// Использование
type User struct {
    ID  uint  `gorm:"primarykey"` Name
    string
    Email string `gorm:"uniqueIndex"`
}

db.AutoMigrate(&User{})
db.Create(&User{Name: "John", Email"john@example.com"})
```

Squirrel — Query Builder:

```
go get github.com/Masterminds/squirrel

query := squirrel.Select("id", "name", "email").
    From("users").
    Where(squirrel.Eq{"age": 25}).
    PlaceholderFormat(squirrel.Dollar)

sql, args, _ := query.ToSql()
```